

BEEVIF Website

Animation & Motion Design Reference Guide

Every animation, transition, and hover effect on the BEEVIF site — the homepage hero slideshow, scroll-triggered reveal/drop-in effects, footer interactions, marquees, and every hover state — explained.

0. Overview

The BEEVIF website does not use any third-party animation library — there is no Framer Motion, GSAP, AOS, React Spring, or React Awesome Reveal in the project's dependencies. Every animation on the site is hand-rolled using three native browser tools:

Technique	How it's used on the site
CSS Transitions	Applied to hover states and to properties (opacity, transform, colour) that flip between two values driven by React state — e.g. a link nudging sideways on hover, or a card fading in once visible.
CSS @keyframes	Defined inline inside component-local <code><style></code> tags. Used for continuously-running effects that aren't simply "state A to state B," such as the hero's crossfade + Ken Burns pan/zoom, and the infinite scrolling marquee strips.
IntersectionObserver	A native browser API that watches an element and fires a callback the moment it scrolls into the viewport. Used to trigger every "reveal on scroll" effect (slide-ins, drop-ins, and the stat counter count-up) exactly once, the first time each element becomes visible.

The sections below are organised by where you'd encounter each effect on the live site: the homepage hero, scroll-reveal effects used across multiple pages, the footer, other hover interactions, and the auto-scrolling marquee strips.

1. Homepage Hero Image Slideshow

Crossfade + “Ken Burns” pan/zoom slideshow

website/components/HeroSlideshow.tsx

TRIGGER: Auto-play every 15s | Also manual via dot buttons

What it looks like: the homepage's large banner image doesn't just "cut" to the next photo — the outgoing photo fades out while the incoming photo fades in on top of it (a crossfade), and at the same time each photo is slowly zooming and drifting (the classic slow pan/zoom effect used in documentaries, known as the "Ken Burns effect"). A row of small dots below the image shows which slide is active and lets a visitor jump to any slide directly.

How it's built: two images are stacked exactly on top of each other, absolutely positioned: the slide that's leaving ("prev") and the slide coming in ("current"). Every time the slideshow advances, React gives the incoming image a brand-new unique key, which forces the browser to treat it as a fresh element and restart its CSS animation from the beginning.

slideFadeIn — animates opacity from 0 to 1 over **1.8 seconds** (ease-in-out), while the outgoing image runs the mirror-image slideFadeOut (opacity 1->0).

kenBurns0 / kenBurns1 — animates transform (a subtle scale + position drift, values pre-defined per photo) over **17 seconds** (ease-in-out), alternating between the two keyframe sets on each cycle so consecutive slides don't all zoom the same direction.

Dot indicators: each dot transitions its width/colour over 500ms; the active dot widens from 8px to 24px and changes to the highlight colour, giving a clear "currently showing" indicator.

Note: this is the only image carousel on the homepage — it is a slow crossfade/zoom slideshow, not a left-right swipe carousel.

2. Scroll-Triggered Reveal Animations

This is the effect behind “images/content falling or dropping in as you scroll.” None of these use a scroll event listener directly — instead every one uses the browser's native **IntersectionObserver**, which watches an element and fires once when it crosses into the visible part of the screen. The observer is disconnected immediately after firing, so each element animates in exactly once and does not repeat if you scroll back up and down again.

The shared pattern used everywhere below:

1. A ref is attached to the element and a boolean *visible* state starts as false.
2. An IntersectionObserver watches the element; once a set percentage of it enters the viewport (the “threshold,” e.g. 0.15 = 15% visible), *visible* flips to true and the observer disconnects.
3. The element's inline style reads that state: while *visible* is false it sits offset and transparent (e.g. `translateY(-70px)`, `opacity: 0`); once true, it animates to `translateY(0)`, `opacity: 1` via a CSS transition (typically 0.6s–0.85s with a custom “overshoot-then-settle” easing curve, `cubic-bezier(0.22,1,0.36,1)`).
4. Where several items reveal together (e.g. a row of cards), each one gets a slightly longer `transitionDelay` than the last (commonly `index × 0.1s` to `0.12s`), producing a staggered cascade rather than every item snapping in simultaneously.

Team & value cards — “drop from above”

`website/components/LeadershipSection.tsx` (About page team cards) ·

`website/components/ValuesSection.tsx` (About page value cards)

TRIGGER: Scroll into view, fires once

The literal “drop in as you scroll” effect: each card starts 70px above its resting position and fully transparent, then falls down into place while fading in — `transform: translateY(-70px) -> translateY(0)` alongside `opacity: 0 -> 1`, over 0.7s (transform) / 0.6s (opacity), staggered by 0.12s per card so the row cascades left-to-right rather than dropping all at once.

Partner logos — drop-in

`website/components/PartnersSection.tsx` (About page)

TRIGGER: Scroll into view, fires once

Same drop-from-above pattern as the team/value cards (`translateY(-70px) -> 0` + fade), staggered by 0.1s per logo.

Section headings — slide in from the sides

`website/components/SectionHeading.tsx` (reused across nearly every page section)

TRIGGER: Scroll into view, fires once

The small eyebrow label slides in from the **left** (`translateX(-120px) -> 0`) while the main heading slides in from the **right** (`translateX(120px) -> 0`) — the two converge toward the centre. Any supporting text underneath simply fades in (opacity only). All three are staggered a fraction of a second apart (0s, 0.08s, 0.18s) over 0.8s.

Mission / Volunteer / Work-programme rows — text & image slide together

website/components/MissionSection.tsx (Home) · VolunteerSection.tsx (Get Involved) · WorkProgrammes.tsx (Our Work, 6 rows)

TRIGGER: Scroll into view, fires once

Two-column layout reveals: the text block slides in from the left ($\text{translateX}(-90\text{px}) \rightarrow 0$) and the accompanying image slides in from the right ($\text{translateX}(90\text{px}) \rightarrow 0$, delayed 0.1s behind the text), both fading in simultaneously, over 0.85s. On Our Work, the six programme rows visually alternate image-left/image-right on desktop, but each element's own slide direction stays fixed relative to itself.

Stat counters — animated count-up

website/components/StatsBar.tsx (Home page)

TRIGGER: Scroll into view, fires once

Rather than a slide/fade, this section's numbers count upward from 0 to their target value once the stat strip scrolls into view, using `requestAnimationFrame` with an ease-out-cubic curve over **1.8 seconds** — giving each statistic a satisfying "ticking up" feel instead of just appearing.

Scroll-to-top button — appear/disappear

website/components/ScrollToTop.tsx

TRIGGER: Plain scroll listener (`window.scrollY > 400`)

Not `IntersectionObserver`-based (it uses a direct scroll event listener instead), and not "fire once" — the button fades and slides up into view ($\text{opacity} + \text{translateY}(16\text{px}) \rightarrow 0$, 0.3s ease) once the visitor has scrolled 400px down, and reverses back out of view if they scroll back up above that point.

3. Footer Animations

Quick Links — slide-out + underline on hover

website/components/Footer.tsx

TRIGGER: Hover

This is the effect specifically requested: each Quick Link in the footer combines **four** hover changes that all run together over 200ms:

- **Slide out** — the link nudges to the right by 8px (translateX), as if sliding out toward the cursor.
- **Underline** — an underline appears beneath the text, offset slightly below the letters for readability.
- **Bold** — the link's font-weight increases to bold.
- **Brighten** — the text colour brightens from a muted 70%-opacity white to solid white.

All four are driven by one Tailwind utility group (transition-all duration-200), so they animate in sync rather than at staggered speeds.

Social icons — opacity fade on hover

website/components/Footer.tsx

TRIGGER: Hover

The social media icons simply dim slightly (fade to 80% opacity) on hover — no movement or underline, just a soft opacity transition.

Contact email link — colour transition

website/components/Footer.tsx

TRIGGER: Hover

Brightens to solid white on hover via a colour transition only — no underline or movement.

The footer itself has no scroll-triggered entrance animation (it does not use IntersectionObserver) — the effects above only ever run on hover.

4. Other Hover Effects Site-Wide

Where	File	Effect
Navbar links (desktop)	Navbar.tsx	Background & text colour transition when active/hovered.
Navbar "Donate" button	Navbar.tsx	Slight dim to 90% opacity on hover.
Mobile hamburger icon	Navbar.tsx	The three bars rotate and shift into an "X" shape when the mobile menu opens (top bar rotates 45°, bottom bar rotates -45°, both shift position) — a CSS-transitioned hamburger-to-close icon morph.
"Ways to Help" cards	WaysToHelpCarousel.tsx	Shadow deepens on hover (flat card lifts visually); CTA buttons dim to 90% opacity.
Volunteer CTA button	VolunteerSection.tsx	Dims to 90% opacity on hover.
Mission "Learn More" link	MissionSection.tsx	Dims to 80% opacity on hover.

Leadership cards	LeadershipSection.tsx	Shadow deepens on hover, in addition to its scroll drop-in entrance.
Partner cards / emails	PartnersSection.tsx	Card shadow deepens on hover; partner email links underline on hover.
Homepage pillar cards	app/page.tsx	Shadow deepens on hover.
Outlined CTA buttons	app/page.tsx (+ other pages)	Border-and-fill inversion — an outlined button fills solid white and its text/border swap colour on hover.
Solid CTA buttons (site-wide)	About / Our Work / Contact / Get Involved	Consistent dim-to-90%-opacity treatment used across nearly every solid call-to-action button on the site, for visual consistency.
Contact page email links	app/contact/page.tsx	Underline appears on hover.
Contact form fields	app/contact/page.tsx	Focus (not hover) ring appears around the active input field.

5. Continuously-Scrolling Marquee Strips

Three sections use an infinite auto-scrolling horizontal strip rather than a static row — the row of content is duplicated back-to-back so that when the animation reaches the halfway point it can loop seamlessly back to the start with no visible jump. Each one pauses in place while the visitor's cursor is hovering over it, then resumes scrolling once the cursor leaves.

Section	File	Loop Speed	Content
Testimonials strip	TestimonialsCarousel.tsx	20s, linear, infinite	Homepage testimonial quote cards
"Ways to Help" strip	WaysToHelpCarousel.tsx	18s, linear, infinite	Get Involved page — ways-to-help cards
Certifications strip	CertificationsSection.tsx	30s, linear, infinite	Our Work page — participant certificate images

All three start moving immediately on page load — they are not gated behind scroll-into-view visibility, unlike the reveal animations in Section 2.

6. Technical Summary

Aspect	Detail
Framework	Next.js 16, React 19
Styling	Tailwind CSS v4 (CSS-first config — no <code>tailwind.config.js</code> ; no custom keyframes defined at the theme level)
Animation libraries	None. No Framer Motion, GSAP, AOS, React Spring, React Awesome Reveal, or React Intersection Observer package — every effect is hand-built.
Reveal-on-scroll mechanism	Native IntersectionObserver API, one instance per component instance, disconnected after first fire
Continuous animations	Hand-written CSS <code>@keyframes</code> inside component-local <code><style></code> tags (hero slideshow, marquee strips)
Hover/state animations	Tailwind utility classes (<code>transition-all</code> , <code>transition-opacity</code> , <code>transition-colors</code> , <code>transition-shadow</code>) plus inline style objects driven by React state
Common easing curve	<code>cubic-bezier(0.22, 1, 0.36, 1)</code> — a gentle "overshoot then settle" curve used across nearly every scroll-reveal transform